

SGA Web con PrimeFaces y Jakarta EE – parte 2



📌 Guía: Implementación de PrimeFaces en una Aplicación Java EE – parte 2 🚀

1 Introducción

En esta parte de la guía, implementaremos las funciones de **agregar**, **editar** y **eliminar** registros de personas utilizando **PrimeFaces** en un proyecto **Jakarta EE** basado en **JSF**, **EJB** y **JPA**.

📌 Objetivo:

- Crear una interfaz amigable para manipular registros de personas.
- Usar **AJAX** para mejorar la experiencia del usuario.
- Integrar validaciones y operaciones **CRUD** con **EJB**.

📌 Arquitectura utilizada:

- **Capa Web:** JSF (PrimeFaces), ManagedBean (Controlador)
- **Capa de Negocio:** EJB (PersonaServiceImpl, PersonaService)

- **Capa de Datos:** JPA (PersonaDaoImpl, PersonaDao)
-

2 Modificar el formulario para agregar un id

➡ Modificamos el formulario del archivo listarPersonas.xhtml para agregar un id, sin un id los eventos Ajax que vamos a agregar más adelante pueden no funcionar correctamente, así que es muy importante agregar un id.

✂ **Antes:**

```
<<h:form>
```

✂ **Después:**

```
<h:form id="personasListForm">
```

💡 **Explicación:** Es necesario agregar un id para el manejo correcto de los eventos Ajax y en general el funcionamiento del formulario para eventos como editar o eliminar registros.

3 Agregar una Persona

♦ Modificar `PersonaBean.java`

En el ManagedBean, agregamos el atributo `personaSeleccionada` para gestionar la nueva persona:

```
private Persona personaSeleccionada;

public Persona getPersonaSeleccionada() {
    return personaSeleccionada;
}

public void setPersonaSeleccionada(Persona personaSeleccionada) {
    this.personaSeleccionada = personaSeleccionada;
}
```

✂ **Modificamos `@PostConstruct` para inicializar el objeto `personaSeleccionada`:**

```
@PostConstruct
public void inicializar() {
    this.personas = personaService.listarPersonas();
    this.personaSeleccionada = new Persona();
}
```

}

✦ Creamos el método para agregar personas:

```
public void agregarPersona() {
    this.personaService.registrarPersona(personaSeleccionada);
    this.personas.add(personaSeleccionada);
    this.personaSeleccionada = new Persona();
}
```

✦ Agregamos el método reiniciarPersonaSeleccionada:

```
public void reiniciarPersonaSeleccionada() {
    this.personaSeleccionada = new Persona();
}
```

💡 Explicación:

- Se agrega una nueva persona a la base de datos.
- Se actualiza la lista de personas en la tabla.
- Se reinicia personaSeleccionada para futuras adiciones.

◆ Modificar listarPersonas.xhtml

✦ Agregamos un botón para abrir el formulario de agregar personas y para regresar al index (antes de cerrar el componente `</p:dataTable>`). Se tiene que importar el namespace `xmlns:f="jakarta.faces.core"`:

```
<f:facet name="footer">
    <div style="display:flex; justify-content:center; gap:10px; margin-top:10px;">
        <p:commandButton value="Agregar" actionListener="#{personaBean.reiniciarPersonaSeleccionada}"
            update=":dialogForm" oncomplete="PF('personaDialog').show()" />
        <p:commandButton value="Regresar" action="index" />
    </div>
</f:facet>
```

✦ Agregamos el diálogo emergente para agregar personas (después del componente de formulario):

```
<p:dialog id="personaDlg" widgetVar="personaDialog" header="Agregar Persona" modal="true"
    showEffect="fade" hideEffect="fade" closable="true" width="500px" responsive="true">
```

```
<h:form id="dialogForm">
```

```

<p:outputPanel id="personaDetail">
    <p:panelGrid id="display" columns="2" style="width:100%"
        columnClasses="ui-grid-col-3, ui-grid-col-9">
        <h:outputText value="Nombre" />
        <h:inputText value="#{personaBean.personaSeleccionada.nombre}" />

        <h:outputText value="Apellido" />
        <h:inputText value="#{personaBean.personaSeleccionada.apellido}" />

        <h:outputText value="Email" />
        <h:inputText value="#{personaBean.personaSeleccionada.email}" />

        <h:outputText value="Teléfono" />
        <h:inputText value="#{personaBean.personaSeleccionada.telefono}" />
    </p:panelGrid>
    <div style="text-align:center; margin-top:15px;">
        <p:commandButton value="Agregar" actionListener="#{personaBean.agregarPersona}"
            update=":personasListForm" oncomplete="PF('personaDialog').hide();" />
        <p:commandButton value="Cancelar" onclick="PF('personaDialog').hide();"
            style="margin-left:10px;" />
    </div>

</p:outputPanel>
</h:form>
</p:dialog>

```

💡 Hasta aquí podemos probar el caso de Agregar.

4 Editar una Persona

◆ Modificar `PersonaBean.java`

✂ Creamos el método para manejar la edición. Se debe importar la clase `org.primefaces.event.RowEditEvent`:

```

public void editListener(RowEditEvent<Persona> event) {
    Persona persona = event.getObject();
    personaService.modificarPersona(persona);
}

```

💡 Explicación:

- Permite editar registros directamente en la tabla.

- Llama al método `modificarPersona()` en la capa de servicio.

◆ **Modificar** `listarPersonas.xhtml`

✦ **Hacemos la tabla editable y agregamos la funcionalidad de edición por cada campo (sustituimos todo el código de `dataTable` con este nuevo código, dejamos el facet de footer del `dataTable` al final):**

```
<p:dataTable id="personasDataTable" value="#{personaBean.personas}" var="persona" editable="true"
    rowKey="#{persona.idPersona}" selection="#{personaBean.personaSeleccionada}" selectionMode="single">

    <p:column headerText="Id">
        <h:outputText value="#{persona.idPersona}" />
    </p:column>

    <p:column headerText="Nombre">
        <p:cellEditor>
            <f:facet name="output">
                <h:outputText value="#{persona.nombre}" />
            </f:facet>
            <f:facet name="input">
                <p:inputText value="#{persona.nombre}" />
            </f:facet>
        </p:cellEditor>
    </p:column>

    <p:column headerText="Apellido">
        <p:cellEditor>
            <f:facet name="output">
                <h:outputText value="#{persona.apellido}" />
            </f:facet>
            <f:facet name="input">
                <p:inputText value="#{persona.apellido}" />
            </f:facet>
        </p:cellEditor>
    </p:column>

    <p:column headerText="Email">
        <p:cellEditor>
            <f:facet name="output">
                <h:outputText value="#{persona.email}" />
            </f:facet>
            <f:facet name="input">
                <p:inputText value="#{persona.email}" />
            </f:facet>
        </p:cellEditor>
    </p:column>
```

```

        </f:facet>
    </p:cellEditor>
</p:column>

<p:column headerText="Telefono">
    <p:cellEditor>
        <f:facet name="output">
            <h:outputText value="#{persona.telefono}" />
        </f:facet>
        <f:facet name="input">
            <p:inputText value="#{persona.telefono}" />
        </f:facet>
    </p:cellEditor>
</p:column>

<p:column headerText="Opciones">
    <p:rowEditor />
</p:column>

<p:ajax event="rowEdit" listener="#{personaBean.editListener}" />

```

💡 Explicación:

- `p:cellEditor` convierte las celdas en editables.
- `p:rowEditor` agrega un botón para editar cada fila.
- `p:ajax event="rowEdit"` llama al método `editListener` cuando el usuario edita un registro.

💡 Explicación: `<p:rowEditor />` Permite mostrar las opciones para editar un registro de la tabla.

- ☒ Agrega un botón de edición en cada fila de la tabla.
- ☒ Cuando el usuario hace clic en el botón, la fila se vuelve editable.
- ☒ Al presionar "Enter" o el ícono de ☒ , los cambios se envían al servidor.

💡 Explicación: `p:ajax event="rowEdit"` Permite guardar automáticamente los cambios al editar una fila.

- ☒ Se dispara **cuando el usuario termina de editar** una fila y presiona "Enter" o confirma los cambios.
- ☒ Llama al método `editListener` en `PersonaBean` para **guardar los cambios en la base de datos**.
- ☒ Se actualizan los datos en la vista automáticamente.

💡 Hasta aquí podemos probar el caso de Editar un registro.

5 Eliminar una Persona

♦ Modificar `PersonaBean.java`

✂ Creamos el método para eliminar personas:

```
public void eliminarPersona() {
    this.personaService.eliminarPersona(personaSeleccionada);
    this.personas.remove(this.personaSeleccionada);
    this.personaSeleccionada = null;
}
```

💡 Explicación:

- Elimina la persona seleccionada de la base de datos.
- Se actualiza la lista en la tabla.

♦ Modificar `listarPersonas.xhtml`

✂ Agregamos un menú contextual para eliminar registros (después del cierre del componente `</p:dataTable>`):

```
<p:contextMenu for="personasDataTable">
    <p:menuItem value="Eliminar" update="personasListForm:personasDataTable" icon="ui-icon-close"
        actionListener="#{personaBean.eliminarPersona}" />
</p:contextMenu>
```

💡 Explicación:

- Se activa con clic derecho sobre un registro.
- Llama al método `eliminarPersona()`.
- La tabla se actualiza automáticamente después de la eliminación.

💡 Con esto ya podemos probar todos los casos, incluyendo el caso de eliminar.

6 Código Final

♦ Archivo `listarPersonas.xhtml`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:h="jakarta.faces.html">
```

```
xmlns:f="jakarta.faces.core"
xmlns:p="primefaces">
<h:head>
  <title>Listado de Personas</title>
</h:head>
<h:body>
  <h:form id="personasListForm">

    <h2 style="text-align:center; margin-bottom:15px;">Listado de Personas</h2>

    <p:dataTable id="personasDataTable" value="#{personaBean.personas}" var="persona"
      editable="true" rowKey="#{persona.idPersona}"
      selection="#{personaBean.personaSeleccionada}" selectionMode="single">

      <p:column headerText="Id">
        <h:outputText value="#{persona.idPersona}" />
      </p:column>

      <p:column headerText="Nombre">
        <p:cellEditor>
          <f:facet name="output">
            <h:outputText value="#{persona.nombre}" />
          </f:facet>
          <f:facet name="input">
            <p:inputText value="#{persona.nombre}" />
          </f:facet>
        </p:cellEditor>
      </p:column>

      <p:column headerText="Apellido">
        <p:cellEditor>
          <f:facet name="output">
            <h:outputText value="#{persona.apellido}" />
          </f:facet>
          <f:facet name="input">
            <p:inputText value="#{persona.apellido}" />
          </f:facet>
        </p:cellEditor>
      </p:column>

      <p:column headerText="Email">
        <p:cellEditor>
          <f:facet name="output">
            <h:outputText value="#{persona.email}" />
          </f:facet>
```



```

        <f:facet name="input">
            <p:inputText value="#{persona.email}" />
        </f:facet>
    </p:cellEditor>
</p:column>

<p:column headerText="Telefono">
    <p:cellEditor>
        <f:facet name="output">
            <h:outputText value="#{persona.telefono}" />
        </f:facet>
        <f:facet name="input">
            <p:inputText value="#{persona.telefono}" />
        </f:facet>
    </p:cellEditor>
</p:column>

<p:column headerText="Opciones">
    <p:rowEditor />
</p:column>

<p:ajax event="rowEdit" listener="#{personaBean.editListener}" />

<f:facet name="footer">
    <div style="display:flex; justify-content:center; gap:10px; margin-top:10px;">
        <p:commandButton value="Agregar"
            actionListener="#{personaBean.reiniciarPersonaSeleccionada}"
            update=":dialogForm" oncomplete="PF('personaDialog').show()" />
        <p:commandButton value="Regresar" action="index" />
    </div>
</f:facet>

</p:dataTable>

<p:contextMenu for="personasDataTable">
    <p:menuItem value="Eliminar" update="personasListForm:personasDataTable"
        icon="ui-icon-close" actionListener="#{personaBean.eliminarPersona}" />
</p:contextMenu>

</h:form>

<p:dialog id="personaDlg" widgetVar="personaDialog" header="Agregar Persona"
    modal="true" showEffect="fade" hideEffect="fade" closable="true" width="500px"
    responsive="true">

```

```

<h:form id="dialogForm">
    <p:outputPanel id="personaDetail">
        <p:panelGrid id="display" columns="2" style="width:100%"
            columnClasses="ui-grid-col-3, ui-grid-col-9">
            <h:outputText value="Nombre" />
            <h:inputText value="#{personaBean.personaSeleccionada.nombre}" />

            <h:outputText value="Apellido" />
            <h:inputText value="#{personaBean.personaSeleccionada.apellido}" />

            <h:outputText value="Email" />
            <h:inputText value="#{personaBean.personaSeleccionada.email}" />

            <h:outputText value="Teléfono" />
            <h:inputText value="#{personaBean.personaSeleccionada.telefono}" />
        </p:panelGrid>
        <div style="text-align:center; margin-top:15px;">
            <p:commandButton value="Agregar" actionListener="#{personaBean.agregarPersona}"
                update=":personasListForm" oncomplete="PF('personaDialog').hide();" />
            <p:commandButton value="Cancelar" onclick="PF('personaDialog').hide();"
                style="margin-left:10px;" />
        </div>

    </p:outputPanel>
</h:form>
</p:dialog>

</h:body>

</html>

```

◆ Archivo `PersonaBean.java`:

```

package sga.web;

import java.util.List;
import jakarta.annotation.PostConstruct;
import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Inject;
import jakarta.inject.Named;
import org.primefaces.event.RowEditEvent;
import sga.domain.Persona;
import sga.servicio.PersonaService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

```

```
@Named("personaBean")
@RequestScoped
public class PersonaBean {

    private static final Logger log = LoggerFactory.getLogger(PersonaBean.class);

    @Inject
    private PersonaService personaService;

    List<Persona> personas;

    private Persona personaSeleccionada;

    @PostConstruct
    public void inicializar() {
        //Iniciamos las variables
        this.personas = personaService.listarPersonas();
        log.debug("personas recuperadas en ManagedBean:" + this.personas);
        this.personaSeleccionada = new Persona();
    }

    public void agregarPersona() {
        this.personaService.registrarPersona(personaSeleccionada);
        this.personas.add(personaSeleccionada);
        this.personaSeleccionada = new Persona();
    }

    public void eliminarPersona() {
        this.personaService.eliminarPersona(personaSeleccionada);
        this.personas.remove(this.personaSeleccionada);
        this.personaSeleccionada = null;
    }

    public void editListener(RowEditEvent<Persona> event) {
        Persona persona = event.getObject();
        personaService.modificarPersona(persona);
    }

    public void reiniciarPersonaSeleccionada(){
        this.personaSeleccionada = new Persona();
    }

    public List<Persona> getPersonas() {
        return personas;
    }
}
```

```
}

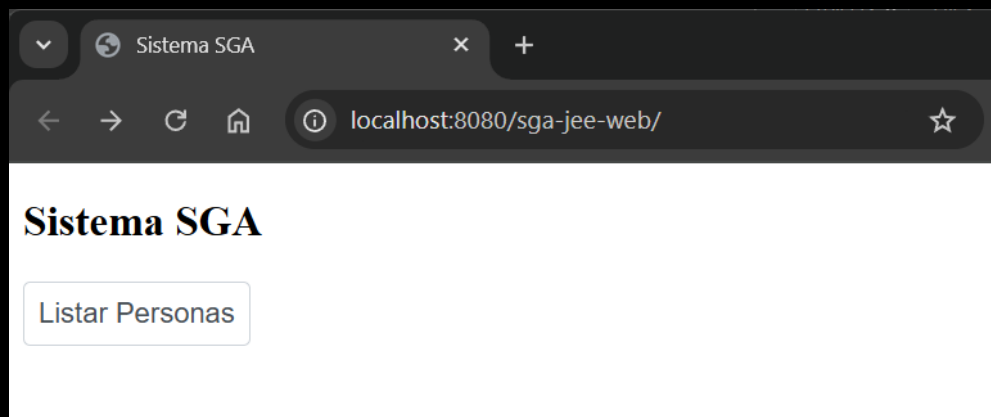
public void setPersonas(List<Persona> personas) {
    this.personas = personas;
}

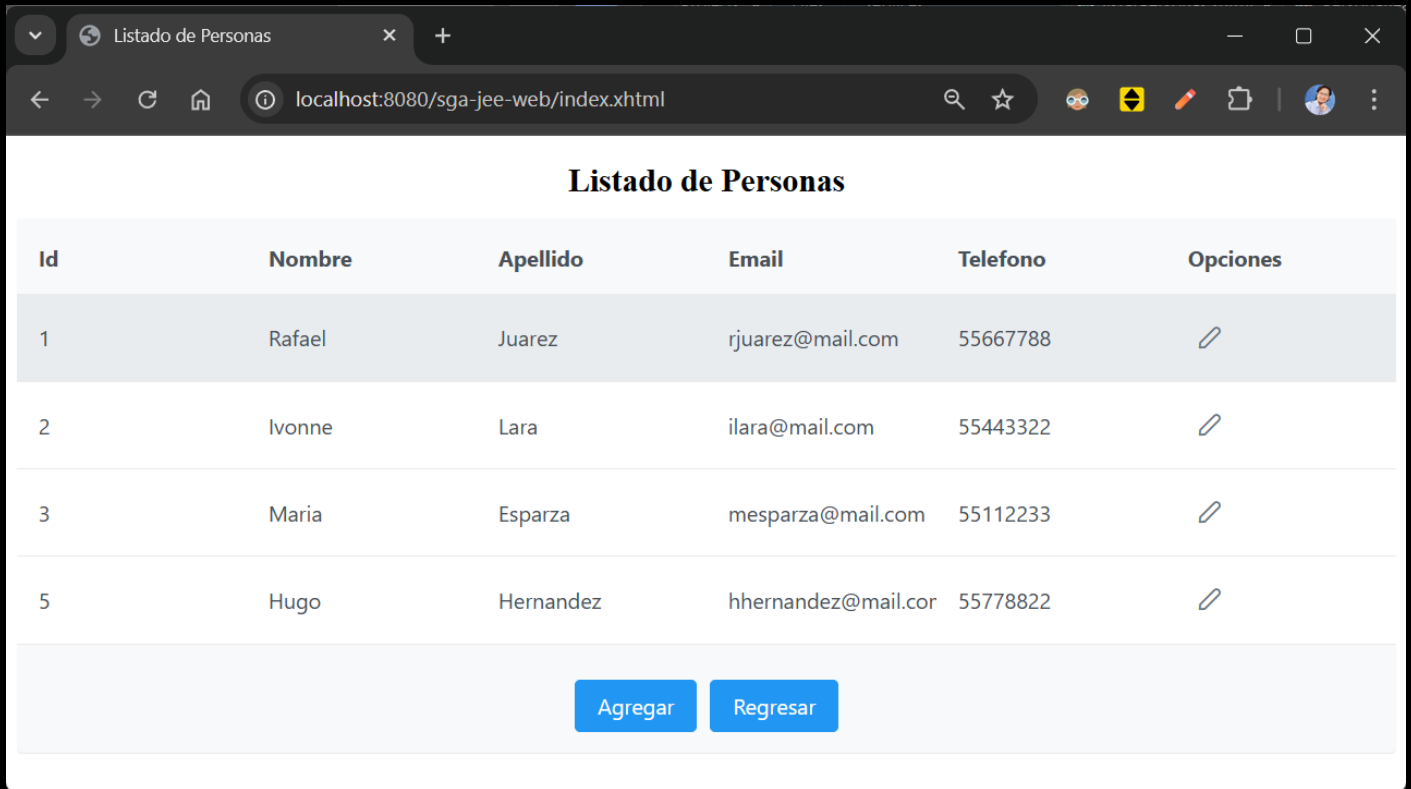
public Persona getPersonaSeleccionada() {
    return personaSeleccionada;
}

public void setPersonaSeleccionada(Persona personaSeleccionada) {
    this.personaSeleccionada = personaSeleccionada;
}
}
```

7 Ejecución

- ✓ Compila y ejecuta el proyecto en GlassFish
- ✓ Accede a: <http://localhost:8080/sga-jee-web/index.xhtml>
- ✓ Prueba las funciones de agregar, editar y eliminar





Id	Nombre	Apellido	Email	Telefono	Opciones
1	Rafael	Juarez	rjuarez@mail.com	55667788	
2	Ivonne	Lara	ilara@mail.com	55443322	
3	Maria	Esparza	mesparza@mail.com	55112233	
5	Hugo	Hernandez	hhernandez@mail.cor	55778822	

AgregarRegresar

Conclusión

¡Hemos implementado con éxito las operaciones **CRUD** en una aplicación Jakarta EE con **PrimeFaces**! 🚀

- ✓ Agregar registros con un formulario emergente.
- ✓ Editar registros directamente en la tabla.
- ✓ Eliminar registros con un clic derecho.
- ✓ Utilizar **AJAX** para mejorar la experiencia de usuario.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)